

SPRAWOZDANIE

Zajęcia: Grafika komputerowa

Prowadzący: Mgr. Inż. Mikołaj Grygiel

Laboratorium 8

Data 11.05.2026 r.

Temat: " Three.js - wieża Hanoi"

Karolina Skiba

62743

Informatyka I

stopień,

stacjonarne,

4 semestr,

Gr.2a

1.Polecenia

Celem projektu było stworzenie interaktywnej, trójwymiarowej wizualizacji gry Wieża Hanoi z wykorzystaniem biblioteki Three.js. Gra miała umożliwić użytkownikowi ręczne przesuwanie krążków pomiędzy słupkami zgodnie z zasadami (mniejszy krążek na większym), a także zawierać funkcje automatycznego rozwiązania oraz podpowiedzi następnego ruchu. Dodatkowym wymaganiem było zaimplementowanie 4 krążków oraz wizualizacji w postaci wielokątnej podstawy (karuzeli).

2.Wykorzystane komendy

2.1 Inicjalizacja sceny, kamery i renderera

```
// --- Inicjalizacja sceny ---
const scene = new THREE.Scene();
scene.background = new THREE.Color(0x0a1030);
scene.fog = new THREE.FogExp2(0x0a1030, 0.006);

const camera = new THREE.PerspectiveCamera(45, window.innerWidth / window.innerHeight, 0.1, 200);
camera.position.set(7, 6, 12);
camera.lookAt(0, 2.5, 0);

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.shadowMap.enabled = true;
renderer.shadowMap.type = THREE.PCFSoftShadowMap;
document.body.appendChild(renderer.domElement);

const labelRenderer = new CSS2DRenderer();
labelRenderer.setSize(window.innerWidth, window.innerHeight);
labelRenderer.domElement.style.position = 'absolute';
labelRenderer.domElement.style.top = '0px';
labelRenderer.domElement.style.left = '0px';
labelRenderer.domElement.style.pointerEvents = 'none';
document.body.appendChild(labelRenderer.domElement);

const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;
controls.dampingFactor = 0.05;
controls.target.set(0, 2.2, 0);
controls.zoomSpeed = 1.2;
```

2.2 Oświetlenie (światło kierunkowe + ambientne)

```

const ambient = new THREE.AmbientLight(0x404060, 0.65);
scene.add(ambient);
const mainLight = new THREE.DirectionalLight(0xffff5e0, 1.2);
mainLight.position.set(5, 12, 4);
mainLight.castShadow = true;
mainLight.shadow.mapSize.width = 1024;
mainLight.shadow.mapSize.height = 1024;
scene.add(mainLight);
const fillLight = new THREE.PointLight(0x5577aa, 0.4);
fillLight.position.set(-2, 3, 4);
scene.add(fillLight);
const backLight = new THREE.PointLight(0xffaa66, 0.35);
backLight.position.set(1, 3, -5);
scene.add(backLight);

const gridHelper = new THREE.GridHelper(15, 20, 0x88aaff, 0x335588);
gridHelper.position.y = -0.25;
gridHelper.material.transparent = true;
gridHelper.material.opacity = 0.35;
scene.add(gridHelper);

```

2.3 Tworzenie krążków (MeshStandardMaterial + geometria cylindra)

```

function getDiskRadius(size) {
    return BASE_RADIUS + (size - 1) * RADIUS_STEP;
}

function createDiskMesh(size, rodX, yPos) {
    const radius = getDiskRadius(size);
    const geometry = new THREE.CylinderGeometry(radius, radius, DISK_HEIGHT, 48, 24);
    const material = new THREE.MeshStandardMaterial({
        color: COLORS[(size-1) % COLORS.length],
        metalness: 0.7,
        roughness: 0.35,
        emissive: 0x331c00,
        emissiveIntensity: 0.08
    });
    const cylinder = new THREE.Mesh(geometry, material);
    cylinder.castShadow = true;
    cylinder.receiveShadow = false;
    cylinder.userData = { type: 'disk', size: size, rodIndex: null };
    cylinder.position.set(rodX, yPos, 0);
    return cylinder;
}

```

2.4 Wielokątna podstawa (ExtrudeGeometry)

```
function createBasePlatform() {
  const group = new THREE.Group();
  const shape = new THREE.Shape();
  const radius = 5.2;
  for (let i = 0; i < 8; i++) {
    const angle = (i / 8) * Math.PI * 2;
    const x = Math.cos(angle) * radius;
    const z = Math.sin(angle) * radius;
    if (i === 0) shape.moveTo(x, z);
    else shape.lineTo(x, z);
  }
}
```

2.5 Raycaster – interakcja myszą (zaznaczanie krążków i słupków)

```
// --- Interakcja myszą (ręczne układanie) ---
const raycaster = new THREE.Raycaster();
const mouse = new THREE.Vector2();
let pegColliders = [];
```

```
function rebuildPegColliders() {
  pegColliders.forEach(c => scene.remove(c));
  pegColliders = [];
  for (let i = 0; i < PEG_COUNT; i++) {
    // Użyj większego collidera dla łatwiejszego klikania
    const colliderGeo = new THREE.CylinderGeometry(0.7, 0.7, 3.5, 8);
    const colliderMat = new THREE.MeshBasicMaterial({ visible: false });
    const collider = new THREE.Mesh(colliderGeo, colliderMat);
    collider.position.set(PEG_POSITIONS[i], 1.8, 0);
    collider.userData = { pegIndex: i, type: 'peg' };
    scene.add(collider);
    pegColliders.push(collider);
  }
}
```

2.6 Algorytm generowania rozwiązań (rekurencyjna Wieża Hanoi)

```
// --- Generowanie optymalnych ruchów (Hanoi) ---
function generateHanoiMoves(n, from, to, via, movesArray) {
  if (n === 1) {
    movesArray.push({ from, to });
  } else {
    generateHanoiMoves(n-1, from, via, to, movesArray);
    movesArray.push({ from, to });
    generateHanoiMoves(n-1, via, to, from, movesArray);
  }
}
```

2.7 Animacja przesuwania krążka (lerp + requestAnimationFrame)

```
// --- Animacja pojedynczego ruchu (płynne przesunięcie) ---
function animateSingleMove(from, to, diskSize, onComplete) {
  if (isAnimatingMove) return;
  const diskEntry = diskItems.find(d => d.rodIdx === from && d.size === diskSize);
  if (!diskEntry) { if (onComplete) onComplete(); return; }
  const mesh = diskEntry.mesh;
  const startPos = mesh.position.clone();
  const targetX = PEG_POSITIONS[to];
  const newY = 0.12 + rods[to].length * DISK_HEIGHT;
  const endPos = new THREE.Vector3(targetX, newY, 0);
  const midPos = new THREE.Vector3((startPos.x + targetX) / 2, 3.8, 0);

  isAnimatingMove = true;
  let startTime = performance.now();
  const duration = 350;

```

```
function step(now) {
  const elapsed = now - startTime;
  let t = Math.min(1, elapsed / duration);
  // ease in-out
  const ease = t < 0.5 ? 2 * t * t : 1 - Math.pow(-2 * t + 2, 2) / 2;
  let currentPos;
  if (t < 0.5) {
    const t2 = t * 2;
    currentPos = startPos.clone().lerp(midPos, t2);
  } else {
    const t2 = (t - 0.5) * 2;
    currentPos = midPos.clone().lerp(endPos, t2);
  }
  mesh.position.copy(currentPos);
  if (t >= 1) {
    mesh.position.copy(endPos);
    isAnimatingMove = false;
    if (onComplete) onComplete();
  } else {
    requestAnimationFrame(step);
  }
}
requestAnimationFrame(step);
}
```

2.8 CSS2DRenderer – etykiety nad słupkami

```
import { CSS2DRenderer, CSS2DObject } from 'three/addons/renderers/CSS2DRenderer.js';
```

```
const label = new CSS2DObject(div);
```

Link do zdalnego repozytorium:

<https://github.com/Linusia/Grafika-komputerowa-2a/tree/main/>

4. Wynik działania



5.Wnioski

- Three.js umożliwia szybkie tworzenie interaktywnych scen 3D w przeglądarce bez znajomości niskopoziomowego WebGL.
- Wykorzystanie raycastera pozwala na precyzyjne wykrywanie kliknięć w obiekty 3D, co jest kluczowe dla interakcji użytkownika.
- BufferGeometry (w tym ExtrudeGeometry) jest wydajniejszym następcą starej klasy Geometry – należy go używać we wszystkich nowych projektach.
- MeshStandardMaterial z parametrami metalness i roughness daje realistyczny, fizycznie poprawny wygląd (PBR) przy odpowiednim oświetleniu.
- Animacje oparte na requestAnimationFrame z interpolacją liniową (lerp) zapewniają płynne przesuwanie obiektów niezależnie od FPS.
- Rekurencyjny algorytm generowania ruchów Hanoi jest efektywny nawet dla większej liczby krążków.
- Kluczowe znaczenie dla UX miało dodanie podświetlenia wybranego krążka (emissive intensity) oraz komunikatów o błędnych ruchach.